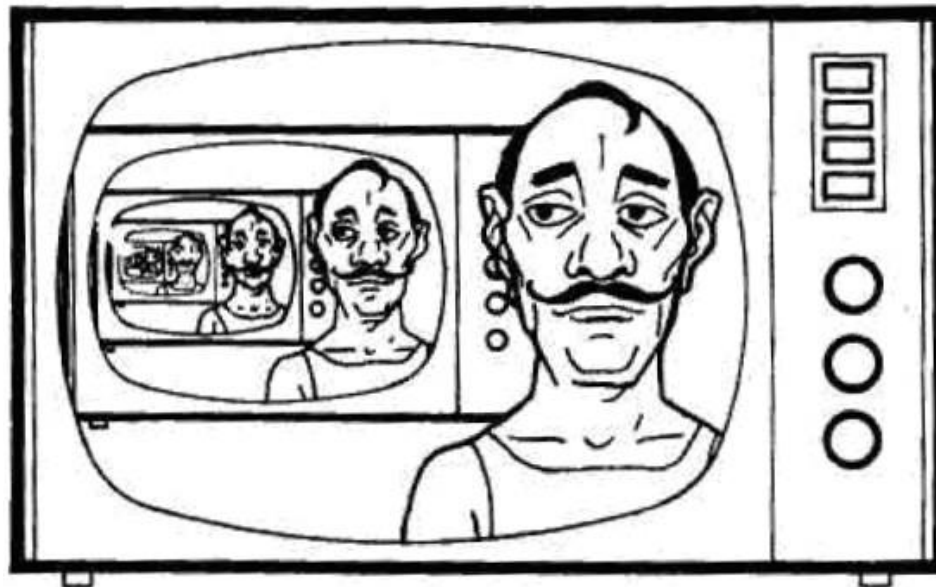


Recursividade

Profa. Rachel Reis
rachel@inf.ufpr.br

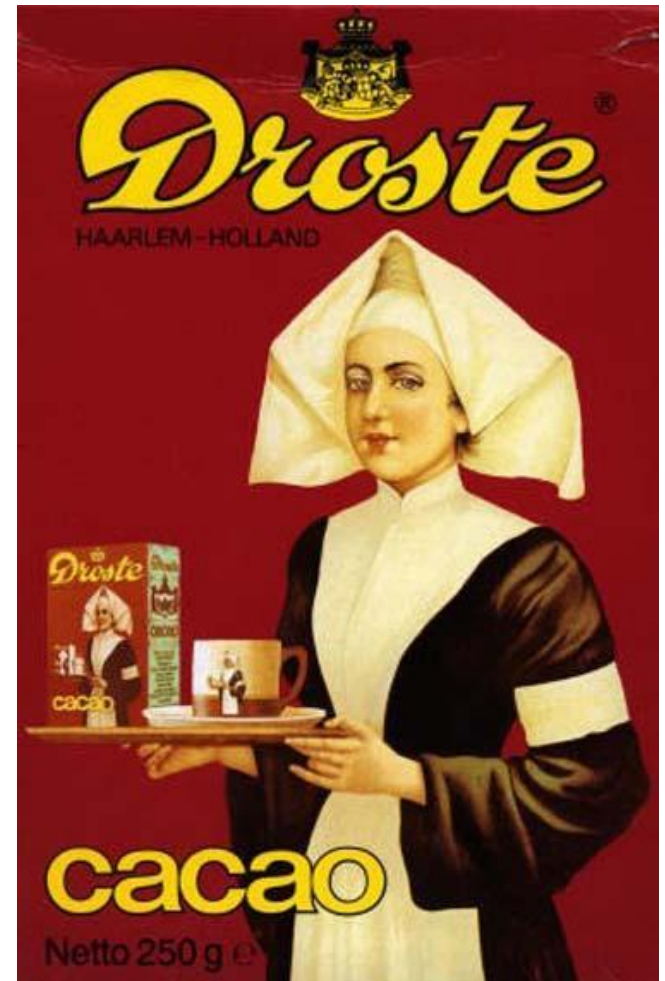
Exemplos de Recursividade

- A ideia de recursividade aparece em outros contextos diferentes da computação.
- No dia a dia.



Exemplos de Recursividade

- Em figuras é usado quando a figura contém ela mesma.
- Isso gera um efeito chamada de efeito “Droste”.



Exemplos de Recursividade

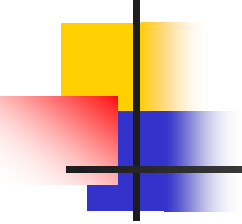
- No Brasil também temos um produto com figura recursiva.



Exemplos de Recursividade

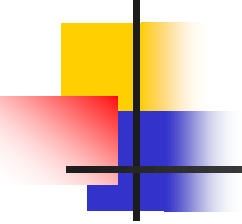
- Sonho recursivo.





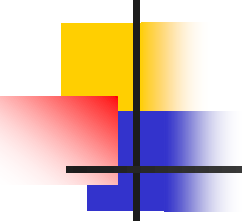
Recursividade na Matemática e Programação

- Uma função é definida recursivamente quando ela chama a si mesma.
- Esse tipo de função é chamada de função recursiva ou relação de recorrência.
- Partes de uma função recursiva:
 - Caso base: encerra a chamada da função recursiva.
 - Passo recursivo: responsável por quebrar o problema em partes menores.



Exemplo 1

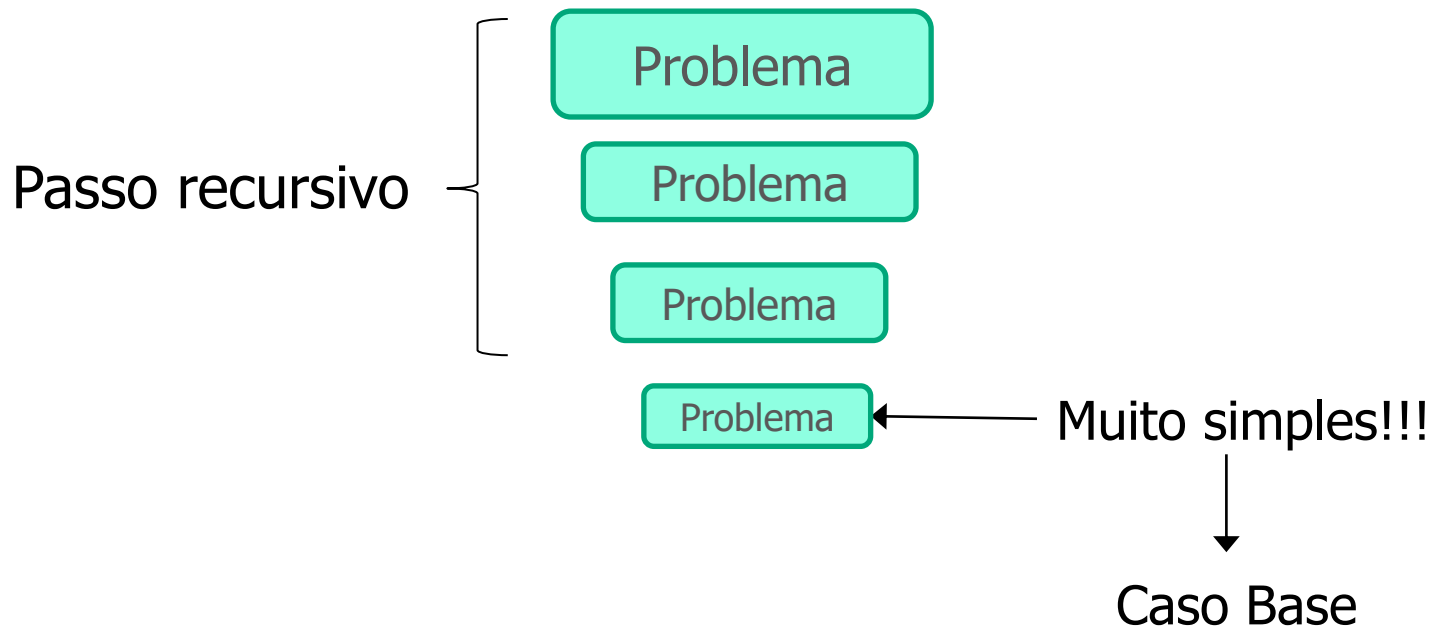
- Implemente uma função recursiva para calcular o **fatorial** de um número inteiro N.
- Partes da função recursiva:
 - Condição de parada (caso base)
 - Passo recursivo

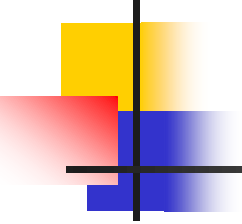


Exemplo 1

- Como se calcula o fatorial do número 3?

$$3! = 3 * 2 * 1 = 6$$





Exemplo 1

- Como se calcula o fatorial do número zero?

$$3! = 3 * 2!$$

$$2! = 2 * 1!$$

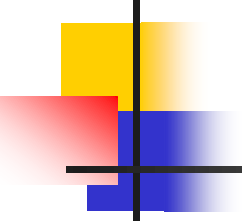
$$1! = 1 * 0!$$

$$0! = 1$$

← **Caso base**

```
if (n == 0)
    return 1;
```

Excepcionalmente, nesta aula: veremos *códigos reais*, e não pseudocódigos.



Exemplo 1

- Como se calcula o fatorial do número 3?

$$3! = 3 * 2!$$

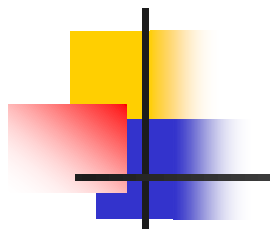
$$2! = 2 * 1!$$

$$1! = 1 * 0!$$

$$0! = 1$$

Passo recursivo

```
else if (n > 0)
    return n * fatorial(n - 1);
```



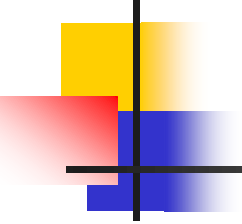
Solução em C e Pascal

Linguagem
C

```
int fatorial(int n){  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

Linguagem
Pascal

```
function fatorial(n: integer): integer;  
begin  
    if n = 0 then  
        fatorial:= 1  
    else if n > 0 then  
        fatorial := n * fatorial(n-1);  
end;
```



Execução

- Como se calcula o **fatorial** do **número 3**?

```
int fatorial(int n) {  
    if(n == 0)                // Caso base  
        return 1;  
  
    else if(n > 0)             // Passo recursivo  
        return n * fatorial(n-1);  
}
```

3!

n

```
int fatorial(int n) {  
    if(n == 0)                // Caso base  
        return 1;  
    else if(n > 0)             // Passo recursivo  
        return n * fatorial(n-1);  
}
```

3!

n

```
int fatorial(int n) {  
    if (n == 0)    
        return 1;  
  
    else if (n > 0)  
        return n * fatorial(n-1);  
}
```


$$\boxed{3!}_n = 3 * \text{fatorial}(2)$$

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\boxed{3!} = 3 * \boxed{2!}$$

n n

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```


$$\boxed{3!} = 3 * \boxed{2!}$$

n n

```
int fatorial(int n){  
    if(n == 0)   
        return 1;  
  
    else if(n > 0)  
        return (n * fatorial(n-1));  
}
```

$$\boxed{\begin{matrix} 3! \\ n \end{matrix}} = 3 * \boxed{\begin{matrix} 2! \\ n \end{matrix}} = \mathbf{2 * fatorial(1)}$$


```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\boxed{\begin{matrix} 3! \\ n \end{matrix}} = 3 * \boxed{2!}$$
$$\boxed{\begin{matrix} 2! \\ n \end{matrix}} = 2 * \boxed{\begin{matrix} 1! \\ \textcolor{red}{n} \end{matrix}}$$

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\boxed{\begin{matrix} 3! \\ n \end{matrix}} = 3 * \boxed{2!}$$
$$\boxed{\begin{matrix} 2! \\ n \end{matrix}} = 2 * \boxed{\begin{matrix} 1! \\ \textcolor{red}{n} \end{matrix}}$$

```
int fatorial(int n) {  
    if (n == 0)    
        return 1;  
  
    else if (n > 0)  
        return n * fatorial(n-1);  
}
```

$$\boxed{3!}_n = 3 * \boxed{2!}_n = 2 * \boxed{1!}_n = \boxed{1!}_n = 1 * \text{fatorial}(0)$$


```
int fatorial(int n) {  
    if (n == 0)  
        return 1;  
    else if (n > 0)  
        return n * fatorial(n-1);  
}
```

$$\begin{aligned} \boxed{3!} &= 3 * \boxed{2!} \\ \boxed{2!} &= 2 * \boxed{1!} \\ \boxed{1!} &= 1 * \boxed{0!} \end{aligned}$$

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\begin{aligned} \boxed{3!} &= 3 * \boxed{2!} \\ \boxed{2!} &= 2 * \boxed{1!} \\ \boxed{1!} &= 1 * \boxed{0!} \end{aligned}$$

```
int fatorial(int n){
    if(n == 0) ← True
        return 1;
    else if(n > 0)
        return n * fatorial(n-1);
}
```

The diagram illustrates the recursive definition of factorial using boxes and arrows. It shows the following sequence of boxes and operations:

- A box containing $3!$ with an arrow pointing to it from a box containing $2!$.
- A box containing $2!$ with an arrow pointing to it from a box containing $1!$.
- A box containing $1!$ with an arrow pointing to it from a box containing $0!$.
- A box containing $0!$ with an arrow pointing to it from a box containing $0!$.

The boxes are arranged in a descending staircase pattern from top-left to bottom-right. The arrows indicate the recursive step: $n! = n * (n-1)!$. The final box contains $0!$ and is labeled with a red 1 and an arrow pointing to it from a box containing $0!$.

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```


$$\begin{array}{c}
 \boxed{3!} \\
 n
 \end{array}
 = 3 * \begin{array}{c} \boxed{2!} \\ n \end{array}
 = 2 * \begin{array}{c} \boxed{1!} \\ n \end{array}
 = 1 * \begin{array}{c} \boxed{0!} \\ n \end{array}$$

$\mathbf{1}$



```
int fatorial(int n) {
```

```
    if(n == 0)
        return 1;
```

Saiu da função

```
    else if(n > 0)
        return n * fatorial(n-1);
```

```
}
```



The diagram illustrates the recursive definition of factorial using boxes and arrows. It shows the following sequence of operations:

- A box containing $3!$ is shown above the variable n .
- An arrow points from this box to the expression $3 *$.
- A box containing $2!$ is shown above the variable n .
- An arrow points from this box to the expression $2 *$.
- A box containing $1!$ is shown above the variable n .
- An arrow points from this box to the expression $1 *$.
- A box containing $0!$ is shown above the variable n .
- An arrow points from this box to the value 1 .

The final result is 1 .

```
int fatorial(int n) {  
    if (n == 0)  
        return 1;  
  
    else if (n > 0)  
        return n * fatorial(n-1);  
}
```

The diagram illustrates the recursive calculation of 3! (3 factorial). It shows the sequence of multiplications:

- $3! = 3 * 2!$
- $2! = 2 * 1!$
- $1! = 1 * 0!$

The final result is 6.

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\begin{array}{c}
 \boxed{3!} \\
 n
 \end{array}
 = 3 * \begin{array}{c} \boxed{2!} \\ n \end{array}
 = 2 * \begin{array}{c} \boxed{1!} \\ n \end{array}$$

1
×

```
int fatorial(int n) {
```

```
    if(n == 0)
        return 1;
```

Saiu da função

```
    else if(n > 0)
        return n * fatorial(n-1);
```

```
}
```



$$\begin{array}{c} \boxed{3!} \\ n \end{array} = 3 * \begin{array}{c} \boxed{2!} \\ n \end{array}$$

$$\begin{array}{c} \boxed{2!} \\ n \end{array} = 2 * \begin{array}{c} \boxed{1!} \\ 1 \end{array}$$

2

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\begin{array}{c} \boxed{3!} \\ n \end{array} = 3 * \begin{array}{c} \boxed{2!} \\ \boxed{2!} \\ n \end{array} = 2 * \boxed{1!}$$

The diagram illustrates the recursive calculation of 3!. It shows that 3! is equal to 3 multiplied by 2!. Below this, it shows that 2! is equal to 2 multiplied by 1!. A red '2' with an arrow points from the '2!' in the second equation to the '2' in the first equation, indicating the recursive step.

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

$$\begin{array}{c}
 \boxed{3!} \\
 n
 \end{array}
 = 3 * \begin{array}{c}
 \begin{array}{c} 2 \\ \boxed{2!} \end{array} \\
 \begin{array}{c} \text{X} \\ \boxed{} \\ n \end{array}
 \end{array}$$

```
int fatorial(int n) {
```

```
    if(n == 0)
```

```
        return 1;
```

Saiu da função

```
    else if(n > 0)
```

```
        return n * fatorial(n-1);
```

```
}
```



$$\boxed{3!} = 3 * \boxed{2!}$$

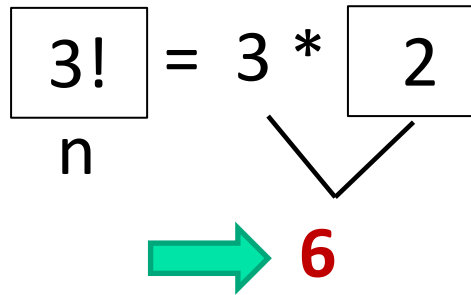
n

2

6

The diagram illustrates the recursive step of calculating a factorial. It shows the equation $3! = 3 * 2!$. The $3!$ is enclosed in a box with the variable n underneath it. The number 3 and the $2!$ are colored blue. Above the $2!$ is the number 2 . Below the equation, the number 6 is shown in red, with two black arrows pointing from the blue 3 and the blue $2!$ to it, indicating that $3 * 2! = 6$.

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```



n

```
int fatorial(int n) {
```

```
    if(n == 0)
```

```
        return 1;
```

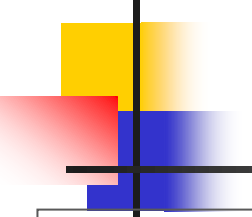
```
    else if(n > 0)
```

```
        return n * fatorial(n-1);
```

```
}
```

Saiu da função





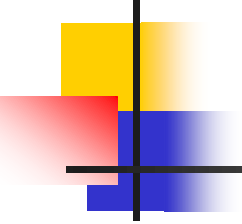
Algoritmo recursivo x iterativo

```
int fatorial(int n){  
    if(n == 0)  
        return 1;  
  
    else if(n > 0)  
        return n * fatorial(n-1);  
}
```

Recursivo

Iterativo

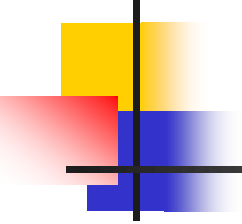
```
int fatorial(int n){  
    int i, fat = 1;  
    for(i = n; i > 0; i--)  
        fat = fat * i;  
    return fat;  
}
```



Fatorial

- Como seria a **função matemática** para representar o código abaixo?

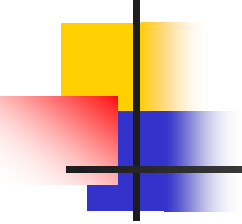
```
int fatorial(int n) {  
    if(n == 0)  
        return 1;  
  
    else if (n > 0)  
        return (n * fatorial(n-1));  
}
```



Função Matemática

$$f(n) = \begin{cases} 1, & n = 0 \\ n \cdot f(n - 1), & n > 0 \end{cases}$$

```
int fatorial(int n){  
    if(n == 0)  
        return 1;  
    else if (n > 0)  
        return (n * fatorial(n-1));  
}
```

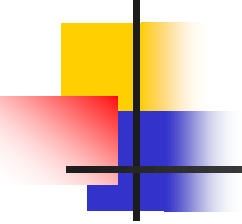


Exemplo 2

- Defina x^n como uma função (matemática) recursiva e, em seguida, traduza para código de computador:

$$x^n = \begin{cases} 1 & \text{se } n = 0 \\ x \cdot x^{n-1} & \text{se } n > 0. \end{cases}$$

Vamos **melhorar** essa representação matemática!



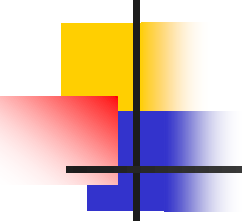
Função Matemática

Nova

$$f(x, n) = \begin{cases} 1, & n = 0 \\ x \cdot f(x, n - 1), & n > 0 \end{cases}$$

Antiga

$$x^n = \begin{cases} 1 & \text{se } n = 0 \\ x \cdot x^{n-1} & \text{se } n > 0. \end{cases}$$



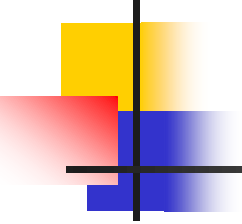
Traduzir para código de computador

$$f(x, n) = \begin{cases} 1, & n = 0 \\ x \cdot f(x, n - 1), & n > 0 \end{cases}$$

→ Caso base

→ Passo recursivo

- Nesse caso, quem seria:
 - Caso base?
 - Passo recursivo?

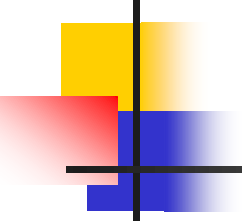


Caso base

$$f(x, n) = \begin{cases} 1, & n = 0 \end{cases}$$

→ Caso base

```
int potencia(int x, int n){  
    if(n == 0)  
        return 1;  
}
```

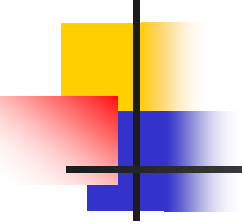


Passo recursivo

$$f(x, n) = \begin{cases} 1, & n = 0 \\ x \cdot f(x, n - 1), & n > 0 \end{cases}$$

Passo recursivo

```
int potencia(int x, int n){  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```



Execução

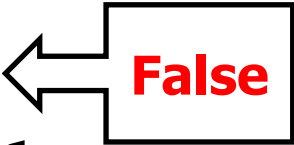
- Como se calcula a potencia de 2^4 ?

```
int potencia(int x, int n){  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

```
int potencia(int x, int n) {  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$P(2, 4) =$

```
int potencia(int x, int n) {  
    if (n == 0)   
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$P(2, 4) =$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 4 - 1)$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

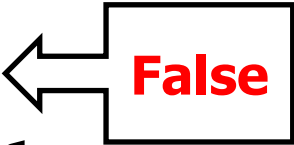
$$P(2, 4) = 2 * P(2, 3)$$

```
int potencia(int x, int n) {  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) =$$


```
int potencia(int x, int n) {  
    if (n == 0)    
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) =$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 3 - 1)$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

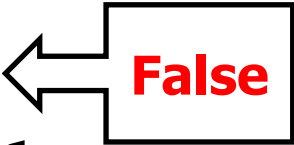
```
int potencia(int x, int n) {  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) =$$

```
int potencia(int x, int n) {  
    if (n == 0)    
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) =$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

```
int potencia(int x, int n){  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1)) ;  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

```
int potencia(int x, int n) {  
    if(n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

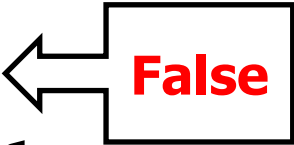
Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

$$P(2, 1) =$$


```
int potencia(int x, int n) {  
    if (n == 0)    
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

$$P(2, 1) =$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1)) ;  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

$$P(2, 1) = 2 * P(2, 1 - 1)$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

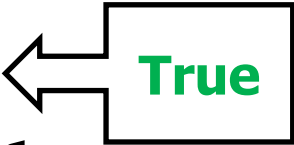
Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

$$P(2, 1) = 2 * P(2, 0)$$

```
int potencia(int x, int n) {  
    if (n == 0)   
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

$$P(2, 1) = 2 * P(2, 0)$$

$$P(2, 0) =$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1)$$

$$P(2, 1) = 2 * P(2, 0)$$

$$P(2, 0) = \mathbf{1}$$

```
int potencia(int x, int n) {  
    if (n == 0) Saiu da função  
        return 1;  
    return (x * potencia(x, n-1));  
}
```



Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1) \quad 1$$

$$P(2, 1) = 2 * P(2, 0)$$

```

int potencia(int x, int n) {
    if (n == 0)
        return 1;
    return (x * potencia(x, n-1));
}

```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2)$$

$$P(2, 2) = 2 * P(2, 1) \quad \xleftarrow{2} \quad 1$$

$$P(2, 1) = \mathbf{2} * \mathbf{P(2, 0)}$$

```
int potencia(int x, int n) {  
    if (n == 0) Saiu da função  
        return 1;  
    return (x * potencia(x, n-1));  
}
```



Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2) \quad 2$$

$$P(2, 2) = 2 * P(2, 1)$$


```

int potencia(int x, int n) {
    if(n == 0)
        return 1;
    return (x * potencia(x, n-1)) ;
}

```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$

$$P(2, 3) = 2 * P(2, 2) \quad \begin{array}{c} \textcolor{red}{4} \\ \swarrow \end{array} \quad 2$$

$$P(2, 2) = \mathbf{2 * P(2, 1)}$$

```
int potencia(int x, int n) {  
    if (n == 0) Saiu da função  
        return 1;  
    return (x * potencia(x, n-1));  
}
```



Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3)$$
$$P(2, 3) = 2 * P(2, 2)$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 * P(2, 3) \quad \begin{array}{c} \textcolor{red}{8} \\ \swarrow \end{array} \quad \begin{array}{c} 4 \\ \downarrow \end{array}$$
$$P(2, 3) = \mathbf{2} * \mathbf{P(2, 2)}$$

```
int potencia(int x, int n) {  
    if (n == 0) Saiu da função  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

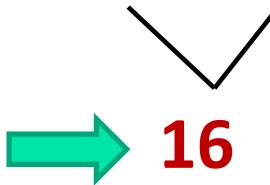


Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2 \overset{8}{*} P(2, 3)$$

```
int potencia(int x, int n) {  
    if (n == 0)  
        return 1;  
    return (x * potencia(x, n-1));  
}
```

Dado $P(x, n)$ calcule a potência de $P(2,4)$:

$$P(2, 4) = 2^8 * P(2, 3)$$


➡ 16

```
int potencia(int x, int n) {  
    if (n == 0) Saiu da função  
        return 1;  
    return (x * potencia(x, n-1));  
}
```



Dado $P(x, n)$ calcule a potência de $P(2, 4)$:

$P(2, 4)$



16

Não é bom pensar dessa forma...

Quando os algoritmos ficarem **mais complexos**
vai ficar mais difícil detalhar até o caso base.

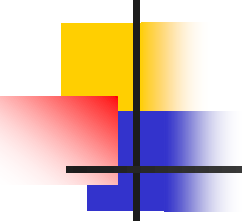


```
int potencia(int x, int n){  
    if(n == 0)  
        return 1;  
  
    return (x * potencia(x, n-1));  
}
```

Recursivo

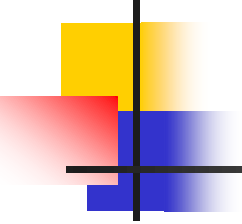
Iterativo

```
int potencia(int x, int n){  
    int result = x;  
    if(n == 0)  
        return 1;  
    while (n > 1){  
        result = result * x;  
        n = n -1;  
    }  
    return result;  
}
```



Vantagens da Recursão

- Código recursivo possui as seguintes vantagens:
 - É mais intuitivo: parece com a definição matemática
 - São menores
 - Aumenta a legibilidade, simplifica a codificação



Passo a Passo para Recursão

- O passo a passo para escrever um código recursivo é:
 1. Escreva o protótipo da sua função: definir as entradas e as saídas.
 2. Escreva o código para o caso base.
 3. Pegue um exemplo concreto e responda: como utilizar um valor *mais próximo* da base (que você já possui) para calcular um valor *mais distante* da base?
 4. Generalize o passo anterior para escrever o código do passo recursivo.

Ex1. (Contagem regressiva). Faça um programa recursivo que receba um inteiro $n \geq 1$ e imprima $n, n - 1, n - 2, \dots, 3, 2, 1$.

1. *Protótipo:*

```
void count_down(int n) {...}
```

2. *Código do caso base:*

```
if (n == 1)
    printf("%d", n);
```

3. *Exemplo concreto:* `count_down(8)`.

Ideia: imprime 8 e chama `count_down(7)`;

4. *Código do passo recursivo:*

```
else{
    printf("%d, ", n);
    count_down(n - 1);
}
```

Ex1. (Contagem regressiva). Faça um programa recursivo que receba um inteiro $n \geq 1$ e imprime $n, n - 1, n - 2, \dots, 3, 2, 1$.

```
void count_down(int n)
{
    if(n == 1) {          // caso base
        printf("%d", n);
    }
    else{                  // passo recursivo
        printf("%d, ", n);
        count_down(n - 1);
    }
}
```

Ex2. (Contagem pra baixo e pra cima). Faça um programa recursivo que receba um inteiro $n \geq 1$ e imprime $n, n-1, \dots, 1, n-1, n$.

1. *Protótipo:*

```
void count_down_up(int n) {...}
```

2. *Código do caso base:*

```
if (n == 1)
    printf("%d", n);
```

3. *Exemplo concreto: count_down_up(4).*

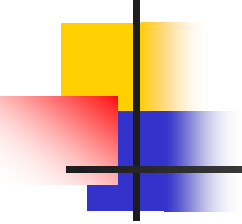
Ideia: imprime 4, chama count_down_up(3), imprime 4

4. *Código do passo recursivo:*

```
else{
    printf("%d, ", n);
    count_down_up(n - 1);
    printf("%d, ", n);
}
```

Ex2. (Contagem pra baixo e pra cima). Faça um programa recursivo que receba um inteiro $n \geq 1$ e imprime $n, n-1, \dots, 1, n-1, n$.

```
void count_down_up(int n)
{
    if(n == 1){
        printf("%d", n);
    }
    else{
        printf("%d, ", n);
        count_down_up(n - 1);
        printf("%d, ", n);
    }
}
```



Exemplo 3

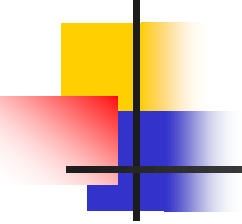
- Sequência de Fibonacci:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144,...

- Propriedade:

- Início da sequência: 0 e 1
- Número subsequente: soma dos dois anteriores

Como calcular o **n-ésimo número** da sequência de Fibonacci?



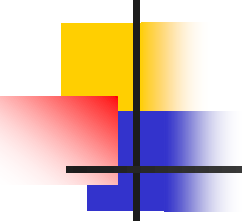
Exemplo 3

- Sequência de Fibonacci:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144,...

- Matematicamente:

$$F(n) = \begin{cases} n & \text{se } n \leq 1 \\ F(n-1) + F(n-2) & \text{se } n > 1 \end{cases}$$

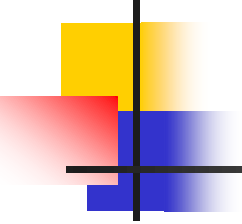


Exemplo 3

$$F(n) = \begin{cases} n & \text{se } n \leq 1 \\ F(n-1) + F(n-2) & \text{se } n > 1 \end{cases}$$

Recursivo

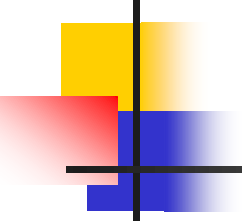
```
int fibonacci(int n) {  
    if (n <= 1)    // Caso base  
        return n;  
  
                    // Passo recursivo  
    return (fibonacci(n-2) + fibonacci(n-1));  
}
```

Exemplo 3

Iterativo

```
int fibonacci(int n){  
    int f1, f2, f3, i;  
    if(n <= 1)  
        return n;  
    f1 = 0;  
    f2 = 1,  
    for(i = 2, i <= n; i++){  
        f3 = f1 + f2;  
        f1 = f2;  
        f2 = f3;  
    }  
    return f3;  
}
```

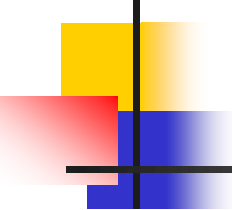


Exemplo 3

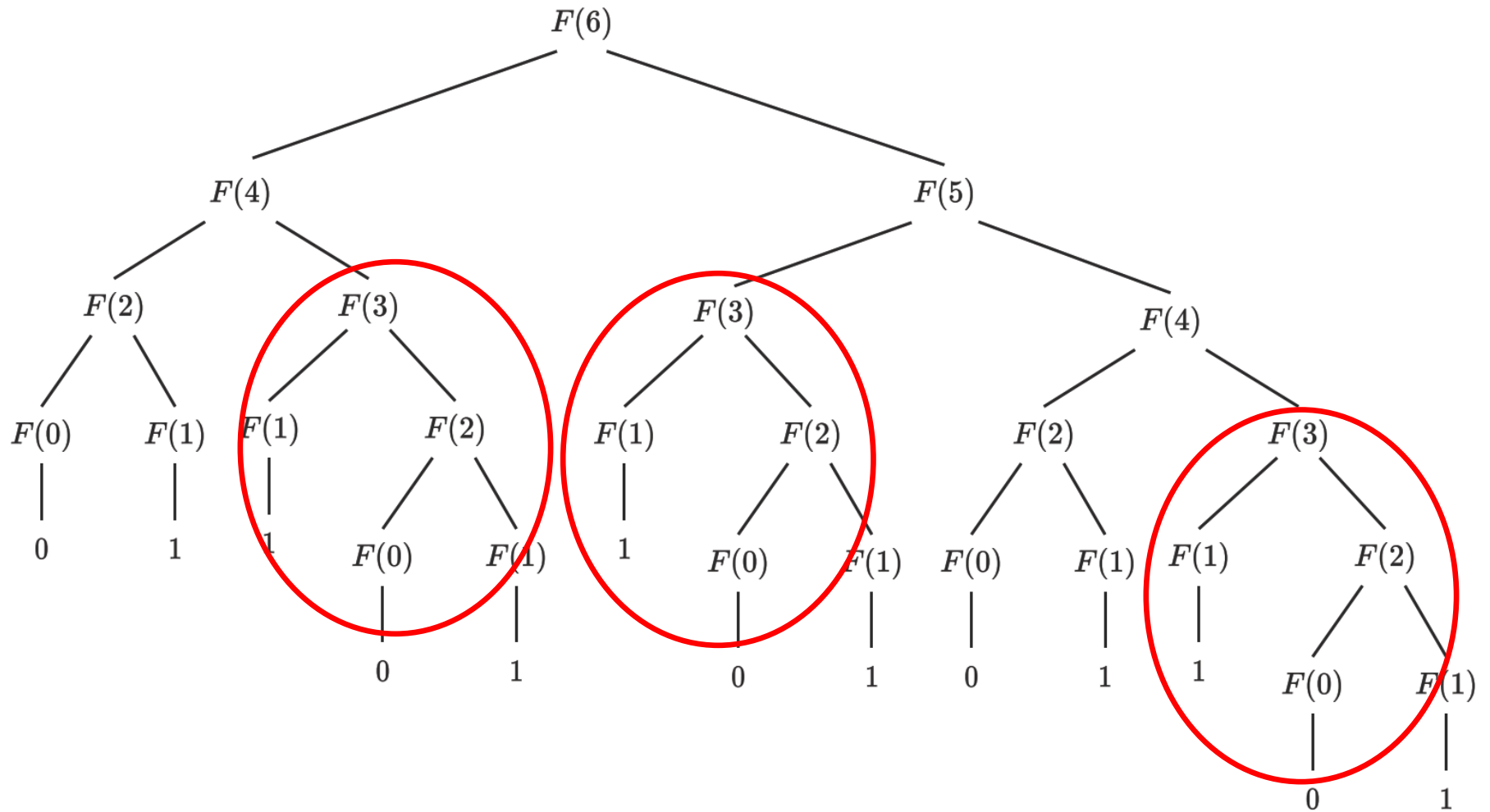
- Solução recursiva:

```
int fibonacci(int n) {  
    if(n <= 1)    // Caso base  
        return n;  
  
                                // Passo recursivo  
    return (fibonacci(n-2) + fibonacci(n-1));  
}
```

- Vantagem:
 - Solução recursiva é mais direta e simples do que a iterativa.
- Desvantagem:
 - Vários cálculos repetidos.



Fibonacci(6)



n	$F(n + 1)$	num. de adições	num. chamadas de função
6	13	12	25
10	89	88	177
15	987	986	1973
20	10946	10945	21891
25	121393	121392	242785
30	1346269	1346268	2692537

Teorema. O número de adições que a versão recursiva de $F(n)$ executa é igual a $F(n + 1) - 1$

- uma **desvantagem** da recursão: chamada de função é custosa